

El Ojo del Árbitro Digital

Se toma un frame al azar del video y se marcan, si aparecen: jugadores en azul, pelota en rojo y líneas de la cancha en verde. Si no aparece ninguno, no se marca nada.

Técnicas usadas: filtrado por color sobre los canales RGB, máscaras y operaciones lógicas, filtro gaussiano, LoG, Canny y Region Growing. La decisión final es una clasificación simple por reglas.

Librerías

```
import numpy as np
import cv2
import os
import random
from datetime import datetime
from collections import deque # cola de píxeles pendientes del Region Growing
import matplotlib.pyplot as plt
# Usamos cv2_imshow porque cv2.imshow no funciona en Colab
from google.colab.patches import cv2_imshow
```

```
def howis(img):
    print('size = ', img.shape)
    print('max = ', np.max(img))
    print('min = ', np.min(img))

def show(images, titles):
    # Armamos una figura con una imagen al lado de la otra
    plt.figure(figsize=(6 * len(images), 4))
    for i, (im, t) in enumerate(zip(images, titles)):
        plt.subplot(1, len(images), i + 1)
        # Si es a color la pasamos de BGR (OpenCV) a RGB (estándar)
        if im.ndim == 3:
            plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        # Si es una máscara la mostramos en escala de grises (blanco = pertenece)
        else:
            plt.imshow(im, cmap='gray')
        plt.title(t)
        plt.axis('off')
    plt.show()
```

Lectura del video desde Drive

```
# Montamos el Drive para poder leer el video
from google.colab import drive
drive.mount('/content/drive')

# Ruta del video dentro del Drive
VIDEO_PATH = '/content/drive/MyDrive/Colab Notebooks/partidoArgVsBrasil_h264.mp4'# Con None sale un frame distin
SEED = None
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force

Adquisición: frame al azar

```
def random_frame(path, seed=None):
    # Abrimos el video y contamos cuántos frames tiene
    video = cv2.VideoCapture(path)
    total = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    # Elegimos un número de frame al azar
    index = random.Random(seed).randint(0, total - 1)
    # Nos paramos en ese frame y lo leemos
    video.set(cv2.CAP_PROP_POS_FRAMES, index)
```

```

ok, frame = video.read()
video.release()
if not ok:
    raise RuntimeError('Could not read frame ' + str(index))
print('Frame', index, 'of', total)
return frame, index

image, index = random_frame(VIDEO_PATH, SEED)
# Verificamos las propiedades de la matriz: alto x ancho x 3 canales
howis(image)
cv2_imshow(image)

```

```

Frame 161783 of 173973
size = (720, 1280, 3)
max = 255
min = 0

```



▼ Umbrales

```

# Todos los umbrales juntos para poder ajustarlos según el video
# Los tamaños están como fracción de la imagen para que no dependan de la resolución
P = dict(
    grass_margin = 12,          # G tiene que superar a R y a B por este valor
    grass_min_g = 50,          # luz verde mínima
    grass_max_b = 150,         # azul máximo del pasto
    grass_max_margin = 45,     # diferencia G - R máxima del pasto
    grass_max_r = 150,         # rojo máximo del pasto
    field_min_fraction = 0.20, # menos de esto de cancha: no es una toma de juego
    white_min = 80,            # los tres canales por encima de esto (la línea fina se mezcla con el pasto)
    white_max_diff = 120,      # y no tan distintos entre sí
    log_threshold = 6,
    canny_low = 60,            # umbrales de histéresis de Canny
    canny_high = 120,
    line_min_length = 0.04,    # respecto del ancho
    line_max_thickness = 12,   # en píxeles
    player_min_height = 0.03,  # respecto del alto
    player_min_area = 0.00015, # respecto del área
    player_min_width = 0.08,   # ancho medio respecto del alto
    player_min_fill = 0.20,    # área / rectángulo que lo encierra
    player_min_ratio = 0.6,    # alto mínimo respecto del ancho
    player_close_passes = 3,   # pasadas para unir las partes de un jugador
    player_margin = 3,         # píxeles que se agranda el contorno
    player_min_contrast = 10,  # desvío mínimo de gris adentro del jugador
    ball_min_area = 0.000008,
    ball_max_area = 0.0006,
    ball_min_circularity = 0.8, # 1 = círculo perfecto
)

```

```

ball_min_brightness = 100,
ball_aspect_range = (0.6, 1.6), # ancho / alto
ball_min_grass_around = 0.6, # fracción de pasto en el borde que rodea a la pelota
# Rectángulos (x0, y0, x1, y1) del logo y el marcador, como fracción de la imagen
sideline_min_length = 0.6, # un lateral cruza al menos esto del ancho
sideline_margin = 5, # píxeles de tolerancia para los pies sobre el lateral
referee_max_value = 70, # oscuro: los tres canales por debajo de esto
referee_min_dark = 0.30, # fracción mínima de oscuro para ser el árbitro
overlay_boxes = [(0.03, 0.04, 0.13, 0.18), (0.83, 0.04, 0.94, 0.19)],
)

```

✓ Kernels y funciones auxiliares

```

# Núcleo gaussiano 3x3 con el factor 1/16: suaviza dando más peso al centro
GAUSS_KERNEL = (1/16) * np.array([[1, 2, 1],
                                   [2, 4, 2],
                                   [1, 2, 1]], dtype=np.float32)

# Núcleo Laplaciano 3x3 de 4 vecinos: da 0 en zonas uniformes
LAPLACE_KERNEL = np.array([[0, 1, 0],
                             [1, -4, 1],
                             [0, 1, 0]], dtype=np.float32)

# Filtro promedio 3x3
MEAN_KERNEL = np.ones((3, 3), dtype=np.float32) / 9

def split_channels(img):
    # Separamos los canales. OpenCV guarda en orden B, G, R
    # Pasamos a int16 para poder restar sin que el resultado se desborde
    B = img[:, :, 0].astype(np.int16)
    G = img[:, :, 1].astype(np.int16)
    R = img[:, :, 2].astype(np.int16)
    return R, G, B

def smooth_mask(mask, times, threshold=0.5):
    # Aplicamos el gaussiano sobre la máscara y volvemos a binaria con un umbral
    # Así se borran los puntos sueltos de ruido
    m = mask.astype(np.float32)
    for _ in range(times):
        m = cv2.filter2D(m, -1, GAUSS_KERNEL)
    return m > threshold

def grow_mask(mask, times):
    # Con el promedio 3x3, si algún vecino era blanco el píxel pasa a blanco
    # La máscara crece 1 píxel por pasada
    m = mask.astype(np.float32)
    for _ in range(times):
        m = cv2.filter2D(m, -1, MEAN_KERNEL)
    return m > 0.01

def close_gaps(mask, times):
    # Agrandamos la máscara y después agrandamos el fondo (NOT) la misma cantidad
    # Se tapan los huecos chicos sin cambiar el tamaño general
    return ~grow_mask(~grow_mask(mask, times), times)

```

✓ Segmentación por color: máscara del pasto

```

def grass_mask(img, p=P):
    R, G, B = split_channels(img)
    # Creamos máscaras binarias con reglas sobre cada canal
    m1 = G > R + p['grass_margin'] # más verde que rojo
    m2 = G > B + p['grass_margin'] # más verde que azul
    m3 = G > p['grass_min_g'] # con suficiente luz
    m4 = B < p['grass_max_b'] # sin mucho azul: la pelota blanca también es verdosa pero tiene B alto
    m5 = G < R + p['grass_max_margin'] # no tan saturado: el buzo flúor del arquero no es pasto
    m6 = R < p['grass_max_r'] # poco rojo: la camiseta amarilla de Brasil tiene R alto
    # El píxel tiene que cumplir todas las condiciones (AND)
    return m1 & m2 & m3 & m4 & m5 & m6

grass = grass_mask(image)
show([image, grass], ['Frame', 'Grass'])

```

Frame



Grass



Region Growing: región de la cancha

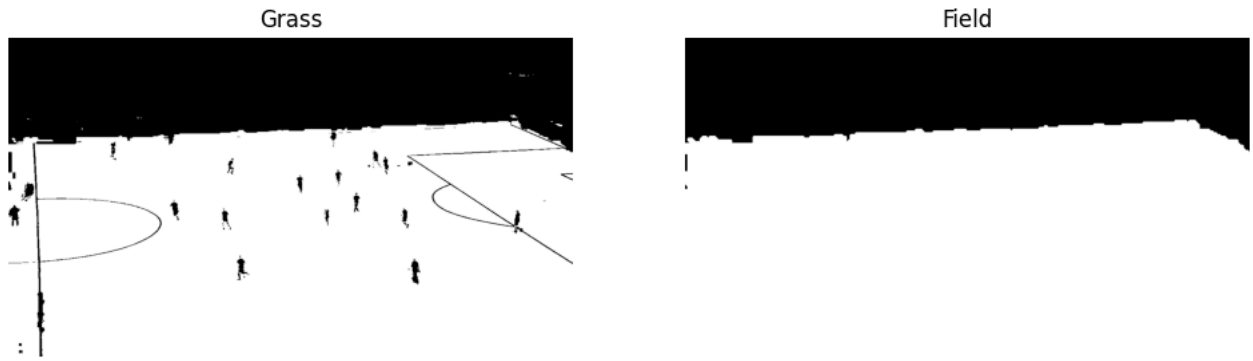
```
def region_growing(mask, seed):
    # Region Growing sobre una máscara binaria: el criterio de similitud es igualdad exacta
    height, width = mask.shape
    region = np.zeros((height, width), dtype=bool)
    pending = deque([seed])
    region[seed] = True
    while pending:
        y, x = pending.popleft()
        # Conectividad 4: arriba, abajo, izquierda y derecha
        for dy, dx in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            ny, nx = y + dy, x + dx
            # Si está dentro de la imagen, es pasto y todavía no está en la región, la región crece
            if 0 <= ny < height and 0 <= nx < width and mask[ny, nx] and not region[ny, nx]:
                region[ny, nx] = True
                pending.append((ny, nx))
    return region

def field_region(grass, p=P):
    height, width = grass.shape
    # Tapamos las líneas (no son pasto y cortarían la cancha en pedazos) y sacamos puntos sueltos
    clean = smooth_mask(close_gaps(grass, 3), 4)
    # Muestreo: tomamos 1 de cada 4 píxeles para que el crecimiento sea más rápido
    small = clean[:, ::4, ::4]

    ys, xs = np.nonzero(small)
    if len(xs) == 0:
        return None
    # Probamos tres semillas abajo (izquierda, centro, derecha) y nos quedamos con la región más grande
    # Así, si una semilla cae en una isla de pasto chica, las otras encuentran la cancha
    region = np.zeros(small.shape, dtype=bool)
    for fx in (0.25, 0.5, 0.75):
        i = np.argmin((xs - small.shape[1] * fx) ** 2 + (ys - small.shape[0] * 0.75) ** 2)
        # Hacemos crecer la región desde la semilla. Así no tomamos el verde de la tribuna o de los carteles
        candidate = region_growing(small, (ys[i], xs[i]))
        if candidate.sum() > region.sum():
            region = candidate
    # Si la región es muy chica el frame no muestra la cancha
    if region.mean() < p['field_min_fraction']:
        return None
    # La cancha llega al borde de abajo o cruza de lado a lado; un cartel LED verde no
    if not (region[-1].any() or (region[:, 0].any() and region[:, -1].any())):
        return None
    # Volvemos al tamaño original con escalamiento
    region = cv2.resize(region.astype(np.uint8), (width, height), interpolation=cv2.INTER_NEAREST)

    # Rellenamos el contorno exterior para que jugadores y líneas queden adentro
    contours, _ = cv2.findContours(region, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    field = np.zeros((height, width), np.uint8)
    cv2.drawContours(field, [max(contours, key=cv2.contourArea)], -1, 1, thickness=-1)
    # La agrandamos un poco para incluir las líneas del borde
    field = grow_mask(field, 3)
    # Sacamos el logo y el marcador de la transmisión, que siempre están en el mismo lugar
    for x0, y0, x1, y1 in p['overlay_boxes']:
        field[int(y0 * height):int(y1 * height), int(x0 * width):int(x1 * width)] = False
    return field

field = field_region(grass)
if field is None:
    print('No field in this frame: nothing is drawn')
else:
    show([grass, field], ['Grass', 'Field'])
```



✓ Líneas de la cancha: color, LoG y Canny

```
def lines_mask(img, grass, field, p=P):
    height, width = grass.shape
    R, G, B = split_channels(img)

    # Blanca: los tres canales altos y parecidos entre sí
    max_channel = np.maximum(np.maximum(R, G), B)
    min_channel = np.minimum(np.minimum(R, G), B)
    white = np.logical_and(min_channel > p['white_min'], (max_channel - min_channel) < p['white_max_diff'])

    # Pasamos a escala de grises con una combinación lineal de los canales
    gray = (0.299 * R + 0.587 * G + 0.114 * B).astype(np.float32)

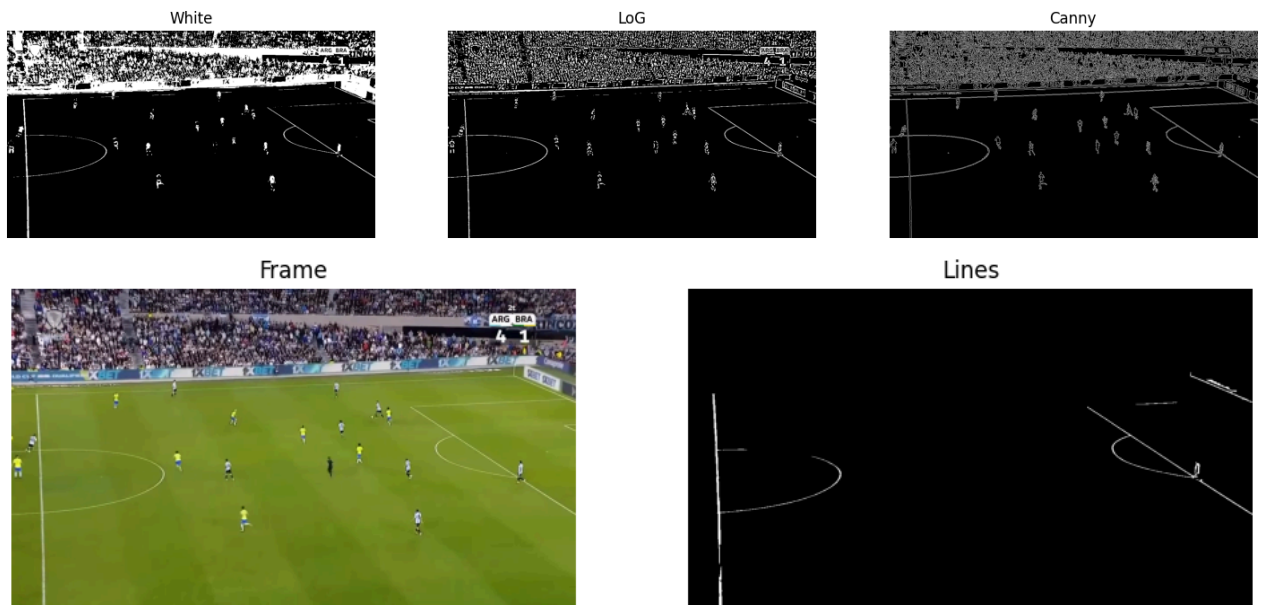
    # LoG: primero suavizamos (3 pasadas para tomar líneas de varios píxeles de ancho)
    smooth = gray
    for _ in range(3):
        smooth = cv2.filter2D(smooth, -1, GAUSS_KERNEL)
    # Después aplicamos el Laplaciano sobre la imagen ya suavizada
    log = cv2.filter2D(smooth, -1, LAPLACE_KERNEL)
    # Negativo = más claro que sus vecinos: el centro de una línea fina
    # En una camiseta blanca el interior da casi 0, así se separan
    thin = log < -p['log_threshold']

    # Canny: la línea tiene que estar cerca de un borde real
    edges = cv2.Canny(gray.astype(np.uint8), p['canny_low'], p['canny_high'])

    # Combinamos todas las condiciones (AND): blanca, fina, con borde, cerca del pasto y en la cancha
    candidate = np.logical_and(white, grow_mask(thin, 2))
    candidate = np.logical_and(candidate, grow_mask(edges > 0, 2))
    candidate = np.logical_and(candidate, grow_mask(grass, 4))
    candidate = np.logical_and(candidate, field)

    # Buscamos el contorno exterior de cada región y también los de sus huecos
    contours, hierarchy = cv2.findContours(candidate.astype(np.uint8), cv2.RETR_CCOMP, cv2.CHAIN_APPROX_NONE)
    lines = np.zeros((height, width), dtype=bool)
    for i, c in enumerate(contours):
        # Si tiene padre es un hueco, no una región
        if hierarchy[0][i][3] != -1:
            continue
        x, y, w, h = cv2.boundingRect(c)
        length = max(w, h)
        # Descartamos las regiones cortas
        if length < p['line_min_length'] * width:
            continue
        # Rellenamos el contorno y borramos sus huecos (ej. el interior del círculo central)
        region = np.zeros((height, width), np.uint8)
        cv2.drawContours(region, [c], -1, 1, thickness=-1)
        for j, hole in enumerate(contours):
            if hierarchy[0][j][3] == i:
                cv2.drawContours(region, [hole], -1, 0, thickness=-1)
        region = np.logical_and(region > 0, candidate)
        # Grosor aproximado: una tira de largo L y grosor g tiene L*g píxeles y perímetro 2*L
        thickness = 2 * region.sum() / max(cv2.arcLength(c, True), 1)
        # Nos quedamos solo con las regiones delgadas
        if thickness < p['line_max_thickness']:
            lines = np.logical_or(lines, region)
    return lines, white, thin, edges

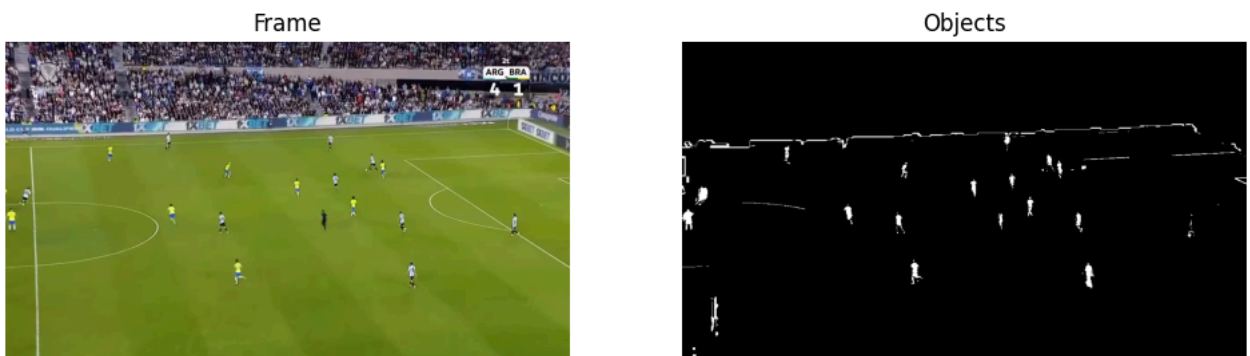
if field is not None:
    lines, white, thin, edges = lines_mask(image, grass, field)
    show([white, thin, edges], ['White', 'LoG', 'Canny'])
    show([image, lines], ['Frame', 'Lines'])
```



Objetos sobre la cancha

```
def objects_mask(grass, field, lines):
    # Lo que está en la cancha y no es pasto ni línea: jugadores, árbitro y pelota
    return np.logical_and(np.logical_and(field, ~grass), ~grow_mask(lines, 1))
```

```
if field is not None:
    objects = objects_mask(grass, field, lines)
    show([image, objects], ['Frame', 'Objects'])
```



Jugadores

```
def detect_players(img, objects, p=P):
    height, width = objects.shape
    # Escala de grises para medir el contraste adentro de cada región
    R, G, B = split_channels(img)
    gray = 0.299 * R + 0.587 * G + 0.114 * B
    # Un jugador aparece partido (camiseta, short, piernas): cerramos huecos y suavizamos para unirlo
    # Con pocas pasadas no se unen dos jugadores que están cerca
    merged = smooth_mask(close_gaps(objects, p['player_close_passes']), 2).astype(np.uint8)
    contours, _ = cv2.findContours(merged, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    players = []
    for c in contours:
        # Medimos cada región: rectángulo que la encierra, área, ancho medio y relleno
        x, y, w, h = cv2.boundingRect(c)
        area = cv2.contourArea(c)
        mean_width = area / max(h, 1)
        fill = area / max(w * h, 1)
        # Es jugador si es alto, tiene área suficiente, es más alto que ancho y no es una tira fina
        if (h >= p['player_min_height'] * height and area >= p['player_min_area'] * height * width
            and h >= p['player_min_ratio'] * w and mean_width >= p['player_min_width'] * h and fill >= p['player_min_ratio']):
            region = np.zeros((height, width), np.uint8)
            cv2.drawContours(region, [c], -1, 1, thickness=-1)
```

```

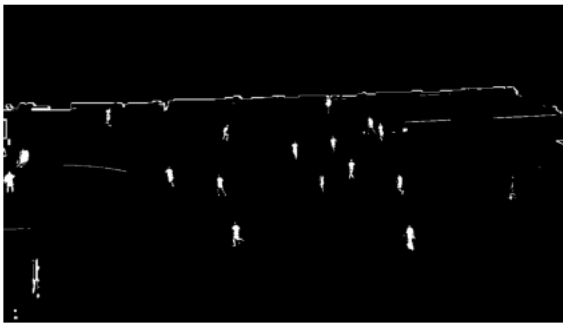
# Un jugador tiene contraste (camiseta, short, piel); una mancha pareja del pasto no
if gray[mp.logical_and(region > 0, objects)].std() < p['player_min_contrast']:
    continue
# Agrandamos un poco la región para que el contorno no corte al jugador
region = grow_mask(region, p['player_margin']).astype(np.uint8)
grown, _ = cv2.findContours(region, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
players.append(grown[0])
return players, merged

if field is not None:
    players, merged = detect_players(image, objects)
# Si dos jugadores están pegados quedan en un solo contorno
print('Players (contours):', len(players))
show([objects, merged], ['Objects', 'Merged'])

```

Players (contours): 17

Objects



Merged



▼ Pelota

```

def detect_ball(img, grass, objects, lines, players, p=P):
    height, width = objects.shape
    R, G, B = split_channels(img)
    # El mínimo de los tres canales indica qué tan blanco es cada píxel
    min_channel = np.minimum(np.minimum(R, G), B)
    near_line = grow_mask(lines, 3)
    # Zona de los jugadores (ya agrandada): un botín blanco suelto no es la pelota
    near_player = np.zeros((height, width), np.uint8)
    cv2.drawContours(near_player, players, -1, 1, thickness=-1)
    near_player = near_player > 0

    # Buscamos el contorno de cada objeto
    clean = smooth_mask(objects, 1).astype(np.uint8)
    contours, _ = cv2.findContours(clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    best, best_score = None, 0
    for c in contours:
        x, y, w, h = cv2.boundingRect(c)
        area = cv2.contourArea(c)
        # Descartamos las muy grandes o muy chicas
        if not (p['ball_min_area'] * height * width <= area <= p['ball_max_area'] * height * width):
            continue
        # Descartamos las que no son casi cuadradas
        if not (p['ball_aspect_range'][0] <= w / max(h, 1) <= p['ball_aspect_range'][1]):
            continue
        # Descartamos las cortadas por el borde del frame
        if x <= 1 or y <= 1 or x + w >= width - 1 or y + h >= height - 1:
            continue
        # Máscara solo con esta región
        region = np.zeros((height, width), np.uint8)
        cv2.drawContours(region, [c], -1, 1, thickness=-1)
        region = region > 0
        # Descartamos las pegadas a una línea (punto del centro y punto penal)
        if np.logical_and(region, near_line).any():
            continue
        # La pelota está rodeada de pasto; una media o un botín están pegados al resto del jugador
        ring = np.logical_and(grow_mask(region, 2), ~region)
        if grass[ring].mean() < p['ball_min_grass_around']:
            continue
        # Descartamos las pegadas a un jugador (botines)
        if np.logical_and(region, near_player).any():
            continue
        # Circularidad = 4 * pi * área / perímetro²: un círculo da 1
        circularity = 4 * np.pi * region.sum() / (cv2.arcLength(c, True) ** 2 + 1e-6)
        brightness = min_channel[region].mean()
        # Si hay varias candidatas nos quedamos con la más redonda y clara
        if circularity >= p['ball_min_circularity'] and brightness >= p['ball_min_brightness']:

```

```

        if circularity * brightness > best_score:
            best, best_score = c, circularity * brightness
        return best

if field is not None:
    ball = detect_ball(image, grass, objects, lines, players)
    print('Ball:', ball is not None)

```

Ball: True

✓ Fuera de la cancha y árbitro

```

def sidelines_mask(lines, p=P):
    height, width = lines.shape
    # Los laterales son las líneas largas que cruzan casi todo el ancho
    contours, _ = cv2.findContours(lines.astype(np.uint8), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    near = np.zeros((height, width), dtype=bool)
    far = np.zeros((height, width), dtype=bool)
    for c in contours:
        x, y, w, h = cv2.boundingRect(c)
        if w < p['sideline_min_length'] * width:
            continue
        region = np.zeros((height, width), np.uint8)
        cv2.drawContours(region, [c], -1, 1, thickness=-1)
        region = np.logical_and(region > 0, lines)
        # En la mitad de abajo es el lateral cercano, en la de arriba el lejano
        if y + h / 2 > height / 2:
            near = np.logical_or(near, region)
        else:
            far = np.logical_or(far, region)
    return near, far

def on_pitch(contour, near, far, p=P):
    # Miramos dónde están los pies: el punto de abajo al centro
    x, y, w, h = cv2.boundingRect(contour)
    fx, fy = x + w // 2, y + h - 1
    near_ys = np.nonzero(near[:, fx])[0]
    far_ys = np.nonzero(far[:, fx])[0]
    # Si los pies quedan debajo del lateral cercano o arriba del lejano, está afuera
    if len(near_ys) and fy > near_ys.max() + p['sideline_margin']:
        return False
    if len(far_ys) and fy < far_ys.min() - p['sideline_margin']:
        return False
    return True

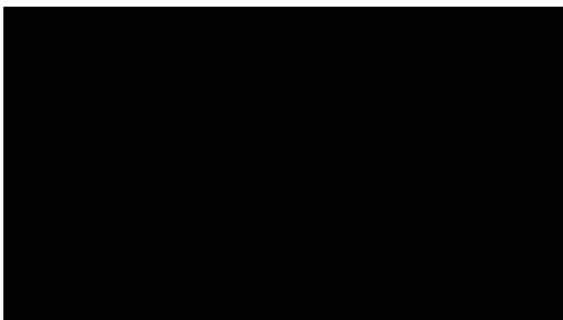
def is_referee(img, contour, p=P):
    height, width = img.shape[:2]
    region = np.zeros((height, width), np.uint8)
    cv2.drawContours(region, [contour], -1, 1, thickness=-1)
    R, G, B = split_channels(img)
    # Oscuro: los tres canales bajos. El árbitro va todo de negro
    dark = np.maximum(np.maximum(R, G), B) < p['referee_max_value']
    return dark[region > 0].mean() > p['referee_min_dark']

if field is not None:
    near, far = sidelines_mask(lines)
    inside = [c for c in players if on_pitch(c, near, far)]
    referees = [c for c in inside if is_referee(image, c)]
    print('Outside the pitch:', len(players) - len(inside), '- Referees:', len(referees))
    show([near, far], ['Near sideline', 'Far sideline'])

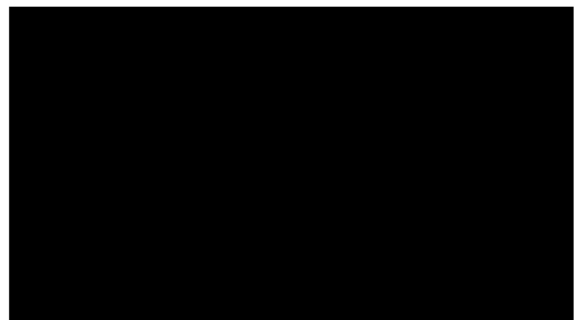
```

Outside the pitch: 0 - Referees: 1

Near sideline



Far sideline



Resultado

```
def process_frame(img, p=P):
    # Trabajamos sobre una copia para no modificar la original
    output = img.copy()
    grass = grass_mask(img, p)
    field = field_region(grass, p)
    # Si no hay cancha no se marca nada
    if field is None:
        return output, dict(lines=False, players=0, referees=0, ball=False)

    lines, _, _, _ = lines_mask(img, grass, field, p)
    objects = objects_mask(grass, field, lines)
    players, _ = detect_players(img, objects, p)
    ball = detect_ball(img, grass, objects, lines, players, p)
    # Sacamos a la gente fuera de la cancha y separamos al árbitro
    near, far = sidelines_mask(lines, p)
    players = [c for c in players if on_pitch(c, near, far, p)]
    referees = [c for c in players if is_referee(img, c, p)]
    players = [c for c in players if not is_referee(img, c, p)]

    # Dibujamos solo lo que se encontró (colores en BGR)
    if lines.any():
        output[grow_mask(lines, 1)] = (0, 255, 0)          # líneas en verde
    if players:
        cv2.drawContours(output, players, -1, (255, 0, 0), 2) # jugadores en azul
    if referees:
        cv2.drawContours(output, referees, -1, (255, 0, 255), 2) # árbitro en magenta
    if ball is not None:
        cv2.drawContours(output, [ball], -1, (0, 0, 255), 2) # pelota en rojo
    return output, dict(lines=bool(lines.any()), players=len(players), referees=len(referees), ball=ball is not None)

result, found = process_frame(image)
print(found)
cv2.imshow(result)
# Guardamos la imagen con la fecha y hora de la corrida, así no se pisa con la anterior
# Queda en la misma carpeta del Drive que el video
file_name = 'result_' + datetime.now().strftime('%Y%m%d_%H%M%S') + '.png'
output_path = os.path.join(os.path.dirname(VIDEO_PATH), file_name)
cv2.imwrite(output_path, result)
print('Image saved to', output_path)
```

```
{'lines': True, 'players': 16, 'referees': 1, 'ball': True}
```

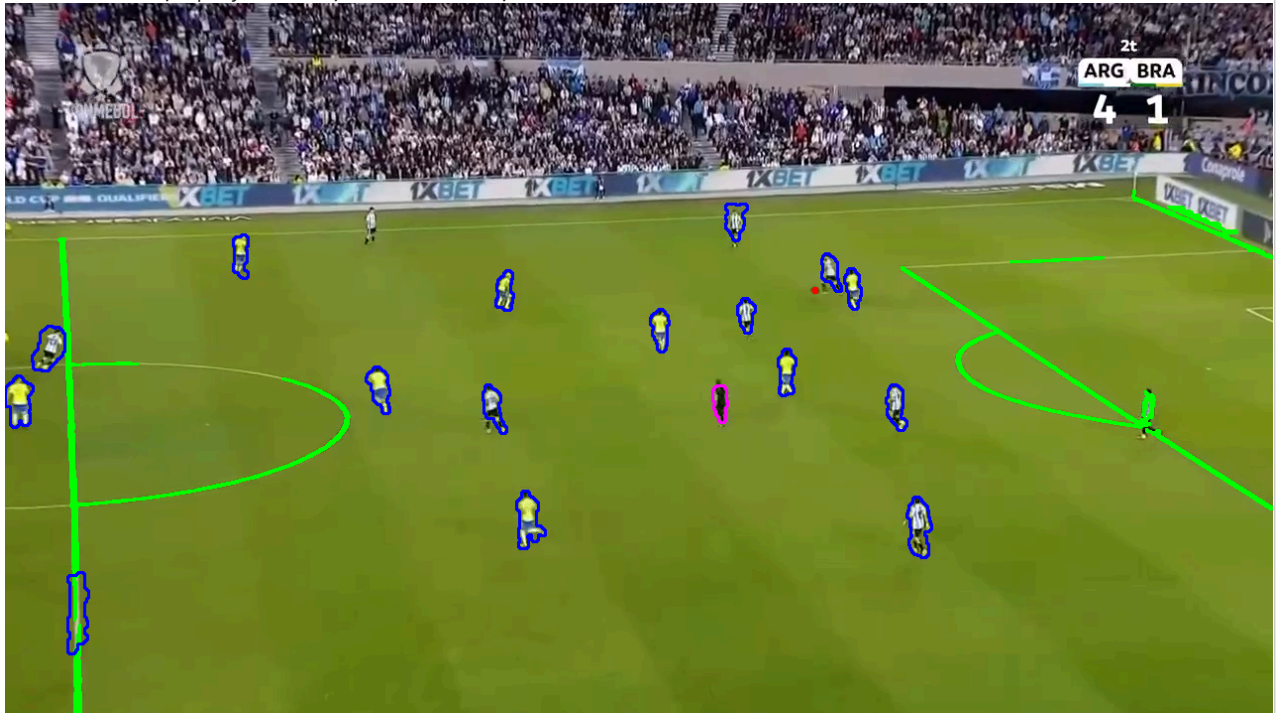


Image saved to /content/drive/MyDrive/Colab Notebooks/result_20260929_004044.png

